CGL: Advancing Continual GUI Learning via Reinforcement Fine-Tuning

Zhenquan Yao[1*], Zitong Huang[1*], Yihan Zeng[2†], Jianhua Han[2], Hang Xu[2], Chun-Mei Feng[3✉*]
Jianwei Ma[1,4✉*], Wangmeng Zuo[1✉*] *

[1]Harbin Institute of Technology [2]Huawei Noah's Ark Lab [3]University College Dublin
[4]Peking University

25s012039@stu.hit.edu.cn, zitonghuang99@gmail.com

Abstract

Graphical User Interface (GUI) Agents, benefiting from recent advances in multimodal large language models (MLLM), have achieved significant development. However, due to the frequent updates of GUI applications, adapting to new tasks without forgetting old tasks in GUI continual learning remains an open problem. In this work, we reveal that while Supervised Fine-Tuning (SFT) facilitates fast adaptation, it often triggers knowledge overwriting, whereas Reinforcement Learning (RL) demonstrates an inherent resilience that shields prior interaction logic from erasure. Based on this insight, we propose a ***C****ontinual* ***G****UI* ***L****earning (CGL) framework that dynamically balances adaptation efficiency and skill retention by enhancing the synergy between SFT and RL. Specifically, we introduce an SFT proportion adjustment mechanism guided by policy entropy to dynamically control the weight allocation between the SFT and RL training phases. To resolve explicit gradient interference, we further develop a specialized gradient surgery strategy. By projecting exploratory SFT gradients onto GRPO-based anchor gradients, our method explicitly clips the components of SFT gradients that conflict with GRPO. On top of that, we establish an AndroidControl-CL benchmark, which divides GUI applications into distinct task groups to effectively simulate and evaluate the performance of continual GUI learning. Experimental results demonstrate the effectiveness of our proposed CGL framework across continual learning scenarios. The benchmark, code, and model will be made publicly available.*

1. Introduction

Graphical User Interface (GUI) agents, powered by Multimodal Large Language Models (MLLMs) [3, 5, 18, 36, 46], have demonstrated significant potential in automating complex interactions within software ecosystems [2, 9, 25–27, 31]. By interpreting visual semantics and executing hierarchical action sequences, these agents bridge the gap between high-level user intent and low-level interface execution. However, the prevailing training paradigm remains largely static, standing in stark contrast to the highly dynamic evolution of real-world GUI environments [6, 21, 31, 37].

Driven by frequent UI updates and functional iterations, GUI agents must possess Continual Learning (CL) capabilities, the ability to adapt to novel interfaces and tasks

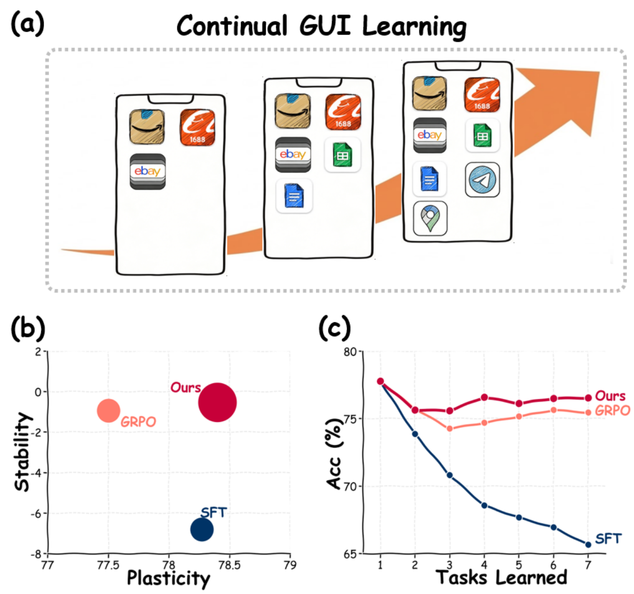

Figure 1. Overview of Continual GUI Learning (CGL). (a) GUI environments evolve as new app categories emerge, causing static GUI agents to misalign with the real world. (b) Existing paradigms struggle to balance stability in legacy tasks and plasticity in novel task accuracy. (c) Our CGL framework achieves balanced adaptation and retention.

[*] These authors contributed equally to this work. ✉ Corresponding authors. † Project leader.

without compromising proficiency in previously mastered domains [13, 29, 35, 47]. However, CL in the GUI domain presents unique challenges distinct from traditional vision or natural language processing [13, 35, 47]. GUI tasks involve exceptionally long-horizon action dependencies, where the integrity of an entire sequence hinges on precise intermediate executions. As shown in Fig.1, exposing an agent to a sequence of evolving applications often results in a precipitous decline in performance on ancestral tasks. Even minor perturbations, such as a subtle shift in login layout or menu hierarchy, can render established interaction policies entirely obsolete.

Existing methods typically employ either Supervised Fine-Tuning (SFT) or Reinforcement Learning (RL), *i.e.*, GRPO [8], to adapt agents to novel tasks [2, 9, 25–27, 31]. However, our investigation reveals a divergence between these two optimization paths. As shown in Fig. 2, the aggressive gradient updates in SFT forcibly pull the model parameters toward the manifold of the new task, thereby distorting the structural integrity of previously acquired knowledge. In contrast, GRPO [8] exhibits a distinct inherent resilience. It preserves broader behavioral diversity, allowing the agent to optimize for rewards without entirely erasing the underlying interaction logic. Nevertheless, this stability is attained at the expense of high sample complexity, resulting in protracted adaptation speeds in unfamiliar environments that fail to meet real-world efficiency requirements.

Motivated by these insights, we propose the *Continual GUI Learning (CGL)* framework, which establishes a synergistic mechanism encompassing conflict quantification and gradient rectification to reconcile the trade-off between adaptation efficiency and skill retention. Specifically, we first introduce *Error-Aware Routing* to dynamically trigger supervised corrective updates when reinforcement exploration fails. To synchronize knowledge acquisition with retention, we develop *Entropy-Regulated Tuning*, which modulates the weight of the supervised objective based on real-time policy uncertainty. Furthermore, we implement *Gradient Surgery* to project exploratory SFT gradients onto a conflict-free subspace defined by GRPO-based anchor gradients, effectively resolving parameter-level interference between the two objectives. This projection process enables selective filtering of update directions, preserving only constructive components that are either orthogonal or aligned with established functional logic. Consequently, CGL empowers the agent with superior exploration efficiency and evolutionary robustness without encroaching upon the "logical redline" of previously acquired skills.

To rigorously benchmark GUI-CL, we establish AndroidControl-CL, a dataset that partitions GUI tasks into sequential groups to simulate realistic software versioning. Our contributions are summarized as follows:

- We reveal that while SFT triggers knowledge overwriting, RL demonstrates inherent resilience in preserving prior GUI interaction logic.
- We propose the CGL framework with entropy-guided SFT balancing and gradient surgery to achieve an effective trade-off between stability and plasticity.
- We introduce AndroidControl-CL, providing a standardized platform for evaluating agent evolution under realistic distribution shifts.
- Extensive experiments demonstrate that CGL significantly outperforms state-of-the-art baselines in both cross-domain adaptation speed and the mitigation of catastrophic forgetting.

2. Related Work

2.1. Continual Learning

Traditional continual learning methods are mainly categorized into three types [24]: Regularization-based methods [1, 20, 41] constrain critical parameter updates via importance quantification, such as adding regularization terms to protect old-task parameters; they are lightweight but face challenges in importance measurement and loss weight balancing. Dynamic Architecture methods [32, 39] freeze old parameters and add task-specific sub-networks, which theoretically avoid forgetting but lead to linear model size expansion with increasing tasks. Rehearsal methods [15, 23] use memory buffers to mix old and new task data during training, and their key challenge lies in balancing buffer size. For Vision-Language Models (VLMs), continual learning faces unique challenges like maintaining cross-modal alignment. Core solutions include Multi-Modal Replay (MMRE) [4, 12, 17, 28, 42]: it adopts explicit or implicit replay, where explicit replay stores old image-text pairs in a buffer and implicit replay generates pseudo pairs, with limitations of storage consumption and dependence on pseudo-data quality respectively. Cross-Modal Regularization (CREG) [11, 40, 44] adds constraint terms to retain old cross-modal associations, and its key issue is balancing constraint intensity. Parameter-Efficient Adaptation (PEA) [7, 10, 14, 19] uses lightweight modules without modifying core parameters to preserve zero-shot capability, but may struggle with complex tasks due to limited adjustable parameters. Recent work [16] proposes a rollout-based Instance Filtering method (RIF-RFT) to enhance the stability and efficiency of continual post-training in VLMs.

2.2. GUI Agents for Interactive Control

GUI Agents automate human-like interactions with GUI systems by perceiving UI elements and executing target actions, with existing works primarily falling into two technical paradigms. SFT serves as a dominant data-driven paradigm, where specialized multimodal models are fine-tuned on large-scale labeled GUI corpora [6, 21, 37] to

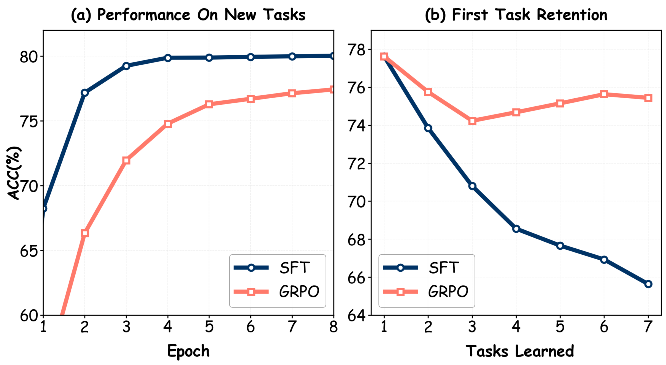

Figure 2. Preliminary comparison of SFT and GRPO on the LLaVA-OneVision-0.5B model. (a) Adaptation performance on a newly introduced task: SFT demonstrates fast plasticity, whereas GRPO exhibits slower adaptation. (b) Forgetting of the initial task after sequential training: SFT undergoes substantial degradation, while GRPO maintains higher retention.

map UI states to actions, as exemplified by UI-TARS [31] which achieves strong benchmark performance but faces generalization limitations. RL-based GUI Agents, inspired by DeepSeek-R1 [8], have gained traction: UI-R1[26] and GUI-G1 [45] explores R1-zero-like visual grounding, while GUI-G2 [34] introduces Gaussian Reward Modeling and CRAFT-GUI [30] proposes a curriculum-reinforced approach to enhance training stability and accuracy. This work [38] dynamically schedules policy entropy to first increase and then decrease during training, enhancing the robustness of GUI agents against environmental noise. Notably, existing research focuses on static task scenarios without addressing the practical need for continuous adaptation to interface and task changes, while our method specifically targets to fill this gap. Meanwhile, existing approaches merely rely on either SFT or RL for optimization, overlooking both their respective strengths and the potential of integrating [22]. To the best of our knowledge, our work presents the first analysis of how SFT, RL, and their integration influence model capabilities within the dynamic setting of continual learning for GUI agent.

3. Task Definition

We formulate Continual GUI Learning (CGL) as a sequence of tasks where a multimodal agent must sequentially master interaction skills across disjoint application domains while preserving previously acquired capabilities. Formally, let $\mathcal{T} = \{T_1, T_2, \ldots, T_N\}$ denote a sequence of N tasks arriving chronologically. Each task T_k comprises a set of mobile applications $\mathcal{A}_k = \{a_{k,1}, a_{k,2}, \ldots, a_{k,M_k}\}$, where applications across distinct tasks are strictly non-overlapping: $\mathcal{A}_i \cap \mathcal{A}_j = \emptyset$ for all $i \neq j$. This ensures that each task introduces a novel application domain unseen in previous tasks.

Within each application $a_{k,m} \in \mathcal{A}_k$, the agent interacts through trajectories. A trajectory $\tau = (I, \{v_t, a_t\}_{t=1}^{T})$ consists of:

- a natural language instruction I specifying the target goal,
- a sequence of visual observations $v_t \in \mathbb{R}^{H\times W\times C}$ representing GUI screenshots at timestep t,
- a corresponding sequence of actions $a_t \in \mathcal{A}_{\text{GUI}}$ executed by the agent, where $\mathcal{A}_{\text{GUI}}$ denotes the unified action space (*e.g.* coordinate-based taps, swipes, and text inputs).

For task T_k, we define a training set $\mathcal{D}_k^{\text{train}}$ and a test set $\mathcal{D}_k^{\text{test}}$, both composed of trajectories sampled from applications in $\mathcal{A}_k$.

Continual Learning Protocol The learning process follows a sequential paradigm where model parameters θ are updated iteratively across $\mathcal{T}$. At stage k, the agent initializes with weights θ_{k-1} learned from preceding tasks (θ_0 denotes the initial pre-trained state), then trains exclusively on $\mathcal{D}_k^{\text{train}}$ without any access to historical training data $\{\mathcal{D}_1^{\text{train}}, \ldots, \mathcal{D}_{k-1}^{\text{train}}\}$. The optimization aims to obtain updated parameters θ_k that minimize the current task loss:

$$\theta_k = \arg\min_{\theta} \mathcal{L}(\theta; \mathcal{D}_k^{\text{train}}), \tag{1}$$

while simultaneously preserving proficiency on all previously encountered tasks. After training on T_k, the agent must generalize to trajectories from *all* applications encountered so far, *i.e.* it is evaluated on the cumulative test set $\bigcup_{i=1}^{k} \mathcal{D}_i^{\text{test}}$. This parameter inheritance mechanism under strict data isolation constraints embodies the core challenge of CGL: accumulating new interaction skills without catastrophically forgetting established GUI capabilities.

Metric. We adopt the following three metrics to evaluate the agent's capabilities.

- **Step-wise Accuracy**: For each task, measures the per-step accuracy, *i.e.* the proportion of correctly executed steps across all steps taken.
- **Average Step-wise Accuracy**: The mean value of step-wise accuracy across all evaluated tasks.
- **Trajectory-wise Accuracy**: For each task, measures the proportion of test trajectories in which every step is executed correctly, *i.e.* the ratio of fully correct execution sequences to the total number of trajectories.
- **Average Trajectory-wise Accuracy**: The mean value of trajectory-wise accuracy across all evaluated tasks.
- **Forgetting Measure (FM)**: Quantifies the performance drop on previous tasks after learning new ones. Let $A_{n,k}$ denote the accuracy on task T_k after training on task T_n (where $n \geq k$):

$$FM = \frac{1}{N-1} \sum_{k=1}^{N-1} (A_{N,k} - A_{k,k}) \tag{2}$$

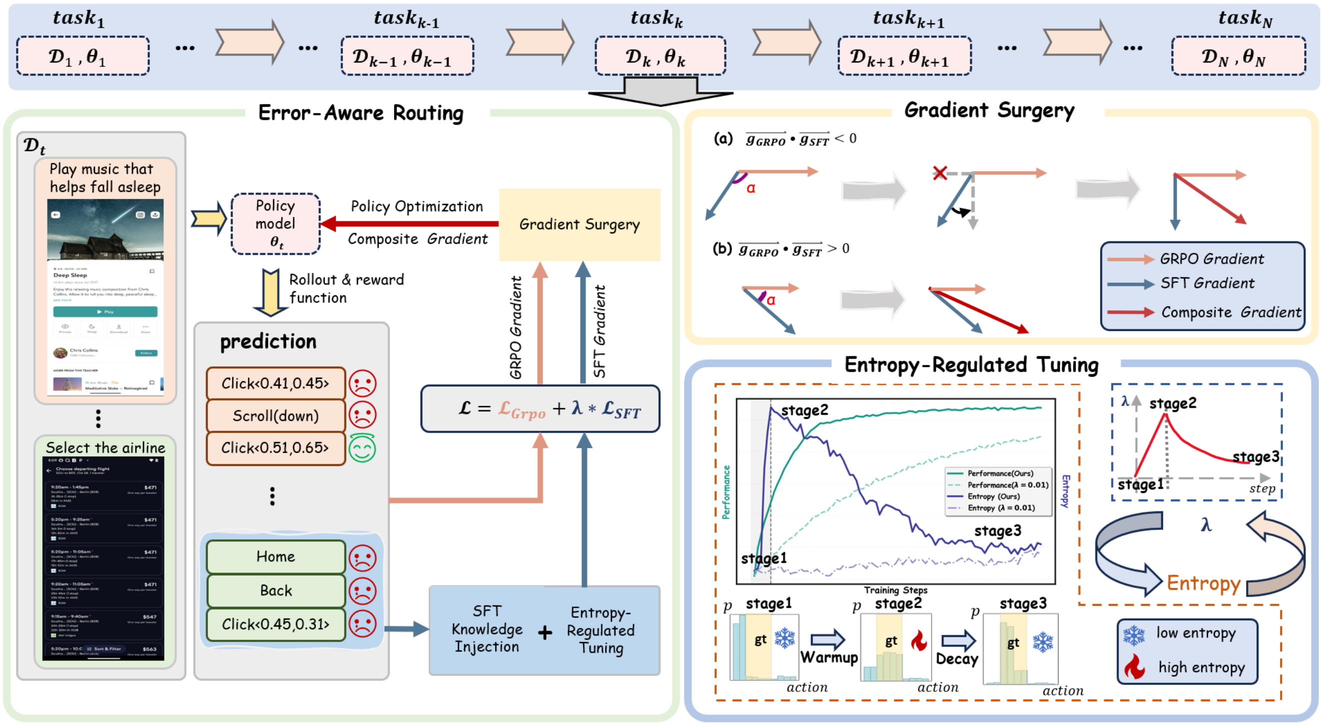

Figure 3. **Overview of the proposed CGL framework. (Left) Error-Aware Routing pipeline:**It dynamically routes data based on prediction feedback, where erroneous actions trigger SFT knowledge injection to facilitate correction. **(Top Right) Gradient Surgery:** Resolves directional conflicts between GRPO and SFT gradients through projection to maintain optimization stability. **(Bottom Right) Entropy-Regulated Tuning:** A dynamic strategy that manages the exploration-exploitation trade-off by adjusting the SFT weight λ. The bottom bar charts illustrate the model's action probability distribution p, where the yellow shaded area denotes the ground truth (gt) action range.

4. Method

Following the established paradigm for GUI agents, our CGL framework is built upon Multimodal Large Language Models (MLLMs), such as LLaVA [18] or Qwen-VL [3, 36]. In this setting, the agent functions as a policy network π_θ that operates over a sequential decision-making process. At each time step t, the model perceives a visual input consisting of the current screenshot v_t and a global human instruction I in natural language. The agent then processes these multimodal inputs to predict the next action a_t in a text-based format, which is subsequently parsed into executable GUI commands. This formulation allows the agent to leverage the strong cross-modal capabilities of pre-trained MLLMs for interface navigation. In preliminary experiments illustrated in Fig. 2, we identified complementary strengths between two training paradigms for GUI agents. **Supervised Fine-Tuning (SFT)** facilitates the efficient acquisition of new knowledge but remains susceptible to the catastrophic forgetting of established tasks. Conversely, **Reinforcement Learning (RL)** demonstrates superior retention of legacy knowledge but frequently fails to converge on new interaction patterns when rewards are sparse.

To capitalize on these distinct advantages, we propose a joint training framework that integrates SFT with Group Relative Policy Optimization (GRPO) [8] illustrated in Fig. 3. Our approach is structured around three core modules designed to stabilize the training process and enhance the agent's adaptability. Specifically, **Error-Aware Routing** addresses the reward sparsity problem by dynamically invoking SFT demonstrations when RL exploration fails. **Entropy-Regulated Tuning** modulates the trade-off between exploration and exploitation by adjusting the weighting factor λ based on policy uncertainty. Finally, **Gradient Surgery** resolves directional conflicts between the exploratory SFT updates and the stabilizing GRPO updates to ensure long-term knowledge retention. The global optimization objective of our framework is formulated as a weighted combination of the two losses:

$$\mathcal{L} = \mathcal{L}_{\text{GRPO}} + \lambda(\mathcal{H}, \text{step}) \cdot \mathcal{L}_{\text{SFT}}, \tag{3}$$

where λ is a function determined by policy entropy $\mathcal{H}$ and training step.

4.1. Error-Aware Routing

To achieve anti-forgetting in continual GUI learning, we adopt GRPO [8] as the RL algorithm, leveraging its group-based relative advantage calculation and constrained policy update. For each GUI query q (screenshot,task instruction,and historical actions), we sample G trajectories from the old policy $\pi_{\theta_{\text{old}}}$, computingnormalized advantages for each trajectory as:$A_{i,t} = \frac{r_i - \mu_r}{\sigma_r + \epsilon}$, where μ_r and σ_r are the mean and standard deviation of rewards across the trajectory group, respectively. The details of the reward function are presented in Sup.2. GRPO optimizes a constrained objective to stabilize policy updates:

$$\begin{aligned}\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}\left[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(O|q)\right] \\ \Bigg[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\Bigg(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q,o_{i,<t})}A_{i,t}, \\ \text{clip}\Bigg(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q,o_{i,<t})}, 1-\epsilon, 1+\epsilon\Bigg)A_{i,t}\Bigg)\Bigg] \\ -\beta \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})]\end{aligned} \tag{4}$$

However, relying solely on GRPO for GUI agent training faces a critical limitation: when the policy has not yet acquired knowledge about a novel interaction pattern, all rollouts for a given instruction remain incorrect throughout execution. Since none of these trajectories achieve the full reward, the relative advantage estimation degenerates—providing no discriminative signal to guide the policy toward correct behaviors. Consequently, the agent becomes trapped in unproductive exploration, severely hindering its ability to acquire new GUI skills.

To break this deadlock, we introduce an **Error-Aware Routing** mechanism that dynamically injects supervised guidance when RL signals become ineffective. Specifically, for each instruction I, we examine the rewards of all K sampled rollouts $\{\tau_k\}_{k=1}^{K}$. If the maximum reward falls short of the ideal full score (i.e., $\max_k r(\tau_k) < r_{\max}$), we interpret this as evidence that the policy cannot autonomously discover the correct solution. In such cases, we route the update to a supervised fine-tuning (SFT) step using the ground-truth demonstration $\tau^* = (I, \{v_t^*, a_t^*\}_{t=1}^{T^*})$, optimizing the likelihood:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|o^*|}\sum_{t=1}^{|o^*|}\log \pi_\theta\left(o_t^* \mid s, o_{<t}^*\right). \tag{5}$$

Particularlly, for spatial actions, we employ an augmented SFT loss by sampling G valid points within the target bounding box to enhance spatial generalization. This routing strategy ensures that the model only utilizes SFT to rectify "pathological" biases where GRPO exploration fails to find a successful path.

4.2. Entropy-Regulated Tuning

To manage the transition from exploration to exploitation across sequential tasks, we introduce a dynamic weighting factor $\lambda(\mathcal{H}, \text{step})$ to modulate the contribution of SFT. The policy entropy, which measures the uncertainty of the agent's action distribution, is defined as $\mathcal{H}(\pi_\theta(\cdot|s)) = -\sum_{a\in\mathcal{A}}\pi_\theta(a|s)\log\pi_\theta(a|s)$,where $\mathcal{A}$ is the action space of the GUI agent.

The mechanics of our collaborative tuning are rooted in the first-order dynamics of this entropy. Let $\mathbf{z}$ denote the policy logits such that $\pi_\theta(a|s) = \text{softmax}(z_a)$. An optimization step induces a logit update $\Delta z_a = \eta\nabla_{z_a}\mathcal{L}$, where η is the learning rate. As derived in our theoretical analysis (see Sup.1), the resulting change in entropy $\Delta\mathcal{H}$ can be approximated by the negative covariance between the current log-probabilities and the logit updates:

$$\Delta\mathcal{H} \approx -\text{Cov}_{a\sim\pi_\theta}\left(\log\pi_\theta(a|s), \Delta z_a\right). \tag{6}$$

Eq. (6) suggests that an update Δz_a positively correlated with the current distribution reinforces confident actions and reduces entropy; conversely, an update that opposes the current bias injects entropy into the policy. Our strategy operates in two distinct phases based on this principle:

Phase 1: Entropy Injection (Warmup). During the initial step$_w$, we linearly increase λ to its maximum. In this stage, the model typically suffers from a *pathological bias* toward incorrect actions, where the probability of the ground-truth action a^* vanishes ($\pi(a^*) \to 0$). The SFT update, $\Delta z_a^{\text{SFT}} \propto (\mathbb{I}[a = a^*] - \pi_\theta(a|s))$, assigns a large positive update to the low-probability target a^* and negative updates to the high-probability erroneous actions. This creates a strong **negative covariance**, acting as an **entropy injector** ($\Delta\mathcal{H}_{\text{SFT}} > 0$). This process "heats up" the distribution, breaking local minima and forcing the agent to explore the ground-truth space.

Phase 2: Entropy Decay (Convergence). Once basic task competency is established, we trigger a decay phase where λ follows an exponential function of $\mathcal{H}$:

$$\lambda(\mathcal{H}) = (\lambda_{\max} - \lambda_{\min})\min\left(1, ke^{\gamma\mathcal{H}}\right) + \lambda_{\min}. \tag{7}$$

In this phase, GRPO updates ($\Delta z_a^{\text{GRPO}} \propto \pi_\theta(a|s)A(s,a)$) dominate the optimization. Due to the **Matthew Effect** (see Sup.1.2), GRPO reinforces actions that already possess high probabilities and positive advantages. This induces a **positive covariance** that drives entropy down ($\Delta\mathcal{H}_{\text{GRPO}} < 0$). By decaying λ as $\mathcal{H}$ drops, we ensure that the SFT component does not interfere with GRPO's precise convergence, allowing the model to solidify its knowledge for long-term retention.

4.3. Conditional Gradient Surgery for Conflict Resolution

During collaborative training of Supervised Fine-Tuning (SFT) and GRPO (a reinforcement learning-based objective), gradient conflicts between the two losses can destabilize optimization and impair knowledge preservation. To address this, we introduce a *conditional* gradient surgery strategy: the SFT gradient is modified *only when* it conflicts with the GRPO gradient's anti-forgetting direction; otherwise, it remains unchanged.

Conflict Detection Criterion We define a conflict as occurring when the cosine similarity between the two gradients is negative (i.e., the angle exceeds $90°$):

$$\cos\alpha = \frac{\nabla_\theta \mathcal{L}_{\text{SFT}} \cdot \nabla_\theta \mathcal{L}_{\text{GRPO}}}{\|\nabla_\theta \mathcal{L}_{\text{SFT}}\|_2 \, \|\nabla_\theta \mathcal{L}_{\text{GRPO}}\|_2} < 0. \tag{8}$$

If $\cos\alpha \geq 0$, the gradients are deemed compatible, and the original SFT gradient is used directly.

Orthogonal Projection under Conflict When a conflict is detected ($\cos\alpha < 0$), we surgically remove the component of the SFT gradient that opposes the GRPO direction. Specifically, we retain only the part of $\nabla_\theta \mathcal{L}_{\text{SFT}}$ orthogonal to $\nabla_\theta \mathcal{L}_{\text{GRPO}}$:

$$\nabla_\theta \mathcal{L}_{\text{SFT}^*} = \nabla_\theta \mathcal{L}_{\text{SFT}} - \nabla_\parallel, \tag{9}$$

where the conflicting parallel component is:

$$\nabla_\parallel = \left(\frac{\nabla_\theta \mathcal{L}_{\text{SFT}} \cdot \nabla_\theta \mathcal{L}_{\text{GRPO}}}{\|\nabla_\theta \mathcal{L}_{\text{GRPO}}\|_2^2} \right) \nabla_\theta \mathcal{L}_{\text{GRPO}}. \tag{10}$$

This projection eliminates the adversarial update direction while preserving all SFT information orthogonal to GRPO's objective. The final update gradient for SFT is thus:

$$\nabla_\theta \mathcal{L}_{\text{SFT}}^{\text{final}} = \begin{cases} \nabla_\theta \mathcal{L}_{\text{SFT}^*}, & \text{if } \cos\alpha < 0 \quad \text{(conflict)}, \\ \nabla_\theta \mathcal{L}_{\text{SFT}}, & \text{otherwise} \quad \text{(no conflict)}. \end{cases}$$

5. Experiment

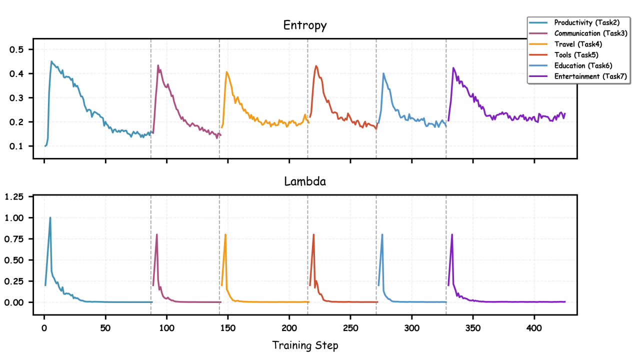

Figure 4. Trends of policy entropy (top) and SFT loss weight λ (bottom) across training steps for Entropy-Regulated Tuning.

5.1. Experimental Setup

Models We adopt two MLLMs with different scales as our base models: *QwenVL2.5-3b-Instruct* [3] and *LLaVA-OneVision-0.5b* [18]. These models are selected to validate the generality of our method across models with varying capacities.

Dataset The dataset used in our experiments is the self-proposed *Android-CL* dataset, which is partitioned into 7 app-related tasks. The task categories include Shopping, Communication, Productivity, Travel, System Tools, Education And Science, and Life And Entertainment. Please refer to the Appendix for detailed task distributions, annotation protocols, and data statistics. To ensure fair and reliable continual-learning evaluation, we define three training orders representing diverse sequential task permutations:

1. Order 1: SP → PO → CO → TT → ST → ES → LE
2. Order 2: SP → LE → ES → ST → TT → CO → PO
3. Order 3: ES → LE → TT → SP → CO → PO → ST

Implementation Details For fair comparison, all methods begin with SFT on the initial task to establish a shared baseline. Subsequent tasks are learned sequentially under their respective continual-learning strategies. Evaluation on the full test sets of all seven tasks assesses both adaptation and forgetting. This design is consistent with existing GUI-agent training pipelines, where many approaches either use SFT alone or apply RL after SFT [2, 9, 25–27, 31]. All experiments are conducted with PyTorch on 8×Ascend 910B NPUs. For SFT and SFT with replay of histrocal data experiments, we adopt the ms-swift framework [43] with a batch size of 16 and a learning rate of 10^{-5}. For GRPO [8], our proposed CGL method, and RIF-RFT [16], all are implemented based on the verl framework [33] with a batch size of 16, rollout batch size of 512, a learning rate of 10^{-6}, a KL-divergence coefficient of 0.01, and a sampling group size of 8. For our CGL method, the hyperparameters are set as: $\lambda_{\text{max}} = 1$, $\lambda_{\text{min}} = 0$, $H_{\text{max}} = 0.45$, $\text{step}_{\text{w}} = 5$, $\gamma = 20$ and $k = e^{-10}$

5.2. Main Results

In the **Method** section, we propose the *CGL* method, which synergistically integrates SFT and GRPO to balance new-task adaptability and old-task anti-forgetting — a critical challenge for GUI agents that remains underexplored in existing literature. And *RIF-RFT* [16] is a recently proposed framework relevant to large-model continual post-training. To comprehensively validate CGL, we conduct experiments using two vision-language models as backbones: lightweight *LLaVA-OneVision-0.5b* [18] and large-scale *QwenVL2.5-3b-Instruct* [3], with **five baselines** covering diverse technical paradigms:

- *SFT*: Standard supervised fine-tuning
- *SFT+KL*: SFT augmented with KL-divergence constraints which is same as grpo;

Table 1. Performance comparison of LLava-onsevision-0.5b under Task Order 1. All accuracy values are reported as percentages (%).

Method	Shopping	Productivity	Communication	Travel	Tools	Education	Entertainment	Avg. Step-Acc.	Avg. Trajectory Acc.	Avg. FM
SFT	65.64	74.73	74.66	72.00	77.39	64.12	80.08	72.66	14.61	-6.81
SFT+KL	74.16	75.23	76.89	75.49	82.06	66.01	79.29	75.59	22.39	-1.78
SFT+Replay	70.88	76.43	77.23	74.19	78.21	64.37	**80.63**	74.56	20.07	-4.69
GRPO[8]	75.44	78.86	78.01	76.41	81.57	67.93	77.80	76.57	23.85	-0.93
RIF-RFT[16]	**76.53**	74.19	75.99	73.55	81.40	65.91	75.34	74.40	22.17	-0.88
Ours	75.55	**81.32**	**80.73**	**76.88**	**83.41**	**68.90**	78.13	**77.84**	**24.77**	**-0.52**
SFT-Joint-Training	77.83	84.46	83.24	78.41	84.76	69.73	79.58	79.69	28.63	-
Zero-Shot	-	-	-	-	-	-	-	-	-	-

Table 2. Performance comparison of QwenVL2.5-3b under Task Order 1. All accuracy values are reported as percentages (%).

Method	Shopping	Productivity	Communication	Travel	Tools	Education	Entertainment	Avg. Step Acc.	Avg. Trajectory Acc.	Avg.FM
SFT	70.54	79.52	78.90	76.41	83.24	69.24	80.42	76.90	23.53	-5.73
SFT+KL	79.30	83.85	82.32	79.84	85.83	74.08	80.71	80.84	34.69	-1.01
SFT+Replay	74.61	83.35	81.66	79.02	85.61	72.26	82.08	79.80	30.19	-3.11
GRPO[8]	79.63	84.74	**83.43**	81.34	85.74	74.08	81.73	81.53	36.78	-0.62
RIF-RFT[16]	79.53	83.35	80.77	80.25	85.94	73.00	80.25	80.44	32.91	-0.58
Ours	**80.32**	**85.46**	81.99	**82.03**	**88.17**	**75.81**	**82.53**	**82.33**	**38.03**	**-0.02**
SFT-Joint-Training	81.73	86.74	85.75	82.24	88.17	77.00	82.76	83.48	41.66	-
Zero-Shot	48.02	57.71	49.50	52.81	55.87	42.33	48.86	50.73	1.86	-

- *SFT+Replay*: SFT augmented with 5% historical data replay for each new task;
- *RIF-RFT* [16]: A recently proposed data-efficient continual post-training framework;
- *GRPO* [8]: Constrained reinforcement learning for sequential policy updates.

5.2.1. Performance under Task Order 1

Tab.1 and Tab.2 present results under **Task Order 1**. Consistent conclusions across model scales validate CGL's superiority in accuracy and anti-forgetting:

- **State-of-the-Art Accuracy**: CGL achieves the highest average Step-Acc. and Trajectory-Acc. for both models:
  - *QwenVL2.5-3b-Instruct*: 82.33% Step-Acc. and 38.03% Trajectory-Acc., outperforming SFT (76.90%), SFT+KL (80.84%), SFT+Replay (79.80%), GRPO (81.53%), and RIF-RFT (80.44%);
  - *LLaVA-OV-One-Vision-0.5b*: 77.84% Step-Acc. and 24.77% Trajectory-Acc., surpassing all baselines by 1.27–5.18 percentage points (pp) in Step-Acc.
- **Optimal Anti-Forgetting (Minimal FM)**: CGL maintains the smallest FM for both backbones:
  - *QwenVL2.5-3b-Instruct*: Near-zero FM (-0.02), reducing forgetting by 0.99 pp (vs. SFT+KL: -1.01) and 0.60 pp (vs. GRPO: -0.62), with negligible old-task knowledge loss;
  - *LLaVA-OV-One-Vision-0.5b*: FM (-0.52), significantly lower than baselines (SFT: -6.81, GRPO: -0.93, etc.).

5.2.2. Robustness Across Multiple Task Orders

To evaluate the robustness of our method to task sequence variations, Tab. 3 presents results across three distinct task orders.

- CGL consistently leads in average Step Accuracy ($\geq$ 82.33%) and average Trajectory Accuracy ($\geq$ 38.03%) across all orders. It outperforms naive SFT (76.87–77.21%) by 5.40–5.55 percentage points in Step Accuracy and surpasses GRPO (81.44–81.75%) by 0.80-0.86 percentage points, confirming its strong generalizability to diverse task sequences.
- Notably, CGL achieves a **positive FM (+0.13) in Task Order 2** — a rare outcome in continual learning, as it indicates not only no forgetting of old tasks but also slight performance enhancement. In stark contrast, SFT exhibits severe forgetting with FMs ranging from -5.33 to -6.03 across all orders, while GRPO (FM = -0.33 to -0.62) also suffers from unavoidable performance degradation on previously learned tasks. This highlights CGL's unique advantage in balancing new-task adaptation and old-task preservation.

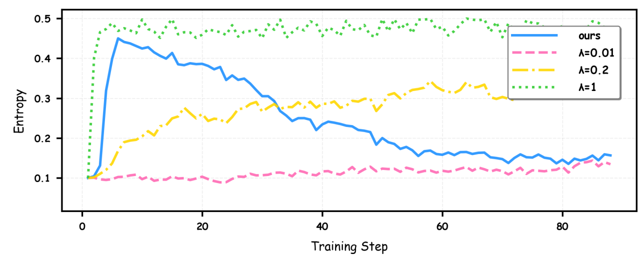

Figure 5. Entropy trends with different hyperparameters

5.2.3. Ablation Study of Module Contributions

Analysis of Key Components. To quantify the individual and synergistic contributions of each component in , we conduct ablation experiments on *QwenVL2.5-3b-Instruct* under Task Order 1, with results presented in Tab. 4. The evaluated modules include SFT, GRPO, KL divergence constraint, dynamic SFT (D-SFT), dynamic SFT weight (D-λ), and gradient surgery (G-Surg).

We first start with the naive SFT baseline, which achieves 76.90% Step-Acc, 23.53% Traj-Acc, and an FM of -5.73. Introducing the KL divergence constraint to SFT improves Step-Acc by 3.94 percentage points (to 80.84%) and reduces FM to -1.01, verifying the basic anti-forgetting effect of KL constraints. Notably, all three metrics of SFT+KL are still inferior to those of *GRPO*, demonstrating that GRPO's superior continual learning capability does

Table 3. Model performance comparison across different task orders.

Model	Task Order 1			Task Order 2			Task Order 3		
	Step Acc. (%)	Trajectory Acc. (%)	FM	Step Acc. (%)	Trajectory Acc. (%)	FM	Step Acc. (%)	Trajectory Acc. (%)	FM
SFT	76.90	23.53	-5.73	77.21	23.46	-5.33	76.87	23.69	-6.03
SFT+KL	80.84	34.69	-1.01	80.77	34.33	-0.96	80.69	34.13	-1.13
SFT+Replay	79.80	27.19	-3.11	80.53	28.06	-2.70	80.39	27.34	-3.29
GRPO[8]	<u>81.53</u>	<u>36.78</u>	-0.62	<u>81.75</u>	<u>36.60</u>	<u>-0.33</u>	<u>81.44</u>	<u>36.72</u>	<u>-0.52</u>
RIF-RFT[16]	80.44	32.91	<u>-0.58</u>	80.50	33.02	-0.46	80.35	31.88	-0.55
Ours	**82.33**	**38.03**	**-0.02**	**82.61**	**38.66**	**+0.13**	**82.40**	**38.57**	**-0.06**

Table 4. Ablation study of modules.

Modules						Metrics		
SFT	**GRPO**	**KL**	**D-SFT**	**D-λ**	**G-Surg**	**Step-Acc**	**Traj-Acc**	**FM**
√						76.90	23.53	-5.73
√		√				80.84	34.69	-1.01
	√	√				81.53	36.78	-0.62
	√					–	–	–
√	√	√				81.68	36.79	-0.57
√	√	√	√			81.90	37.12	-0.33
√	√	√	√		√	82.10	37.57	<u>-0.05</u>
√	√	√	√	√		<u>82.23</u>	<u>37.62</u>	-0.07
√	√	√	√	√	√	**82.33**	**38.03**	**-0.02**

not solely stem from its KL divergence constraint.

The GRPO module requires the KL divergence constraint to operate stably: without KL, GRPO collapses and performance metrics are unavailable. Combining GRPO with KL elevates Step-Acc to 81.53% and Traj-Acc to 36.78%, with FM reduced to -0.62. This demonstrates that constrained reinforcement learning enhances both accuracy and anti-forgetting capability.

We then incrementally integrate additional modules into the SFT+GRPO baseline: Integrating static full-SFT into GRPO boosts Step-Acc to 81.68%. Replacing it with D-SFT further improves Step-Acc to 81.90%, targeting policy weak points to avoid redundant knowledge interference. Incorporating D-λ based on the D-SFT framework further raises Step-Acc to 82.23% and Traj-Acc to 37.62%. This module adjusts SFT's weight via policy entropy and training timesteps, refining the balance between adapting to new tasks and preserving knowledge of old ones. Introducing Grad-Surg boosts Step-Acc to 82.10% and reduces FM to -0.05. This module mitigates gradient conflicts between sequential tasks, preventing performance degradation of previously learned knowledge during new task training. Combining D-λ and Grad-Surg elevates Step-Acc to 82.33% and Traj-Acc to 38.03%, as the two modules complement each other in weight adjustment and gradient stabilization. When all modules are integrated (the full CGL framework), we achieve the optimal performance: 82.33% Step-Acc, 38.03% Traj-Acc, and an FM of -0.02. This result confirms that each module contributes complementary improvements, and their synergistic integration enables CGL to balance new-task accuracy and old-task anti-forgetting effectively.

Analysis of λ As the core of the D-λ module, λ regulates the injection intensity of SFT error samples, and its dynamic adjustment coupled with policy entropy is key to CGL's performance.

Trend Correlation (Fig. 4) depicts the co-variation of policy entropy (top) and λ (bottom) across training steps. At task-switching points (vertical dashed lines), λ rises linearly in the warmup phase to boost entropy introducing new behaviors via error-sample injection, then decays exponentially in the decay phase to stabilize entropy. This "entropy rise-then-decay" pattern balances new-task adaptability and old-task retention.

Table 5. Performance comparison under different λ settings for QwenVL2.5.

QwenVL2.5	Avg.Step-Acc.
$\lambda = 1$	81.62
$\lambda = 0.2$	81.58
$\lambda = 0.01$	81.90
ours	**82.33**

Performance Validation (Tab. 5) and entropy trends (Fig. 5): We compare fixed-λ settings with our dynamic λ (ours): $\lambda = 1$:excessive error-sample injection leads to overly high entropy. $\lambda = 0.01$ insufficient injection maintains low entropy, trapping the model in local optima. Our D-λ (dynamic λ) balances injection intensity across tasks, achieving the highest average score (82.33).

6. Conclusion

In this paper, we studied the *Continual GUI Learning (CGL)* problem, and proposed a standardized benchmark and a collaborative SFT-RL framework to address it. Specifically, we constructed a CGL benchmark by partitioning the AndroidControl dataset into 7 app-category-specific subsets of comparable scale, filling the gap of unified evaluation in this field. To alleviate the stability–plasticity tradeoff between existing methods, we designed two key mechanisms, Entropy-Regulated Tuning and Gradient Surgery for the SFT-RL framework, which balances prior knowledge retention and new skill acquisition. Extensive experiments under three evaluation protocols show that our approach outperforms pure GRPO, pure SFT, and RIF-RFT, demonstrating its effectiveness in the CGL task.

Acknowledgements

This work was supported by the National Key R&D Program of China under Grant No. 2023YFA1008500

References

[1] Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European conference on computer vision (ECCV)*, pages 139–154, 2018. 2

[2] Hao Bai, Yifei Zhou, Jiayi Pan, Mert Cemri, Alane Suhr, Sergey Levine, and Aviral Kumar. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning. *Advances in Neural Information Processing Systems*, 37:12461–12495, 2024. 1, 2, 6

[3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025. 1, 4, 6

[4] Xiang Chen, Jintian Zhang, Xiaohan Wang, Ningyu Zhang, Tongtong Wu, Yuxiang Wang, Yongheng Wang, and Huajun Chen. Continual multimodal knowledge graph construction. *arXiv preprint arXiv:2305.08698*, 2023. 2

[5] Zhe Chen, Weiyun Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Erfei Cui, Jinguo Zhu, Shenglong Ye, Hao Tian, Zhaoyang Liu, et al. Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv preprint arXiv:2412.05271*, 2024. 1

[6] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. Seeclick: Harnessing gui grounding for advanced visual gui agents. *arXiv preprint arXiv:2401.10935*, 2024. 1, 2

[7] Bowen Dong, Zitong Huang, Guanglei Yang, Lei Zhang, and Wangmeng Zuo. Mr-gdino: efficient open-world continual object detection. *arXiv preprint arXiv:2412.15979*, 2024. 2

[8] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 2, 3, 4, 6, 7, 8

[9] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14281–14290, 2024. 1, 2, 6

[10] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022. 2

[11] Linlan Huang, Xusheng Cao, Haori Lu, and Xialei Liu. Class-incremental learning with clip: Adaptive representation adjustment and parameter fusion. In *European Conference on Computer Vision*, pages 214–231. Springer, 2024. 2

[12] Zitong Huang, Ze Chen, Zhixing Chen, Erjin Zhou, Xinxing Xu, Rick Siow Mong Goh, Yong Liu, Wangmeng Zuo, and Chunmei Feng. Learning prompt with distribution-based feature replay for few-shot class-incremental learning. *arXiv preprint arXiv:2401.01598*, 2024. 2

[13] Zitong Huang, Ze Chen, Yuanze Li, Bowen Dong, Erjin Zhou, Yong Liu, Rick Siow Mong Goh, Chun-Mei Feng, and Wangmeng Zuo. Class balance matters to active class-incremental learning. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 9445–9454, 2024. 2

[14] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991. 2

[15] Zhong Ji, Zhishen Hou, Xiyao Liu, Yanwei Pang, and Xuelong Li. Memorizing complementation network for few-shot class-incremental learning. *IEEE Transactions on Image Processing*, 32:937–948, 2023. 2

[16] Song Lai, Haohan Zhao, Rong Feng, Changyi Ma, Wenzhuo Liu, Hongbo Zhao, Xi Lin, Dong Yi, Min Xie, Qingfu Zhang, et al. Reinforcement fine-tuning naturally mitigates forgetting in continual post-training. *arXiv preprint arXiv:2507.05386*, 2025. 2, 6, 7, 8

[17] Stan Weixian Lei, Difei Gao, Jay Zhangjie Wu, Yuxuan Wang, Wei Liu, Mengmi Zhang, and Mike Zheng Shou. Symbolic replay: Scene graph as prompt for continual learning on vqa task. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 1250–1259, 2023. 2

[18] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, et al. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024. 1, 4, 6

[19] Hong Li, Zhiquan Tan, Xingyu Li, and Weiran Huang. Atlas: Adapter-based multi-modal continual learning with a two-stage learning strategy. *arXiv preprint arXiv:2410.10923*, 2024. 2

[20] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017. 2

[21] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19498–19508, 2025. 1, 2

[22] Weiting Liu, Han Wu, Yufei Kuang, Xiongwei Han, Tao Zhong, Jianfeng Feng, and Wenlian Lu. Automated optimization modeling via a localizable error-driven perspective. *arXiv preprint arXiv:2602.11164*, 2026. 3

[23] Yuyang Liu, Yang Cong, Dipam Goswami, Xialei Liu, and Joost Van De Weijer. Augmented box replay: Overcoming foreground shift for incremental object detection. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 11367–11377, 2023. 2

[24] Yuyang Liu, Qiuhe Hong, Linlan Huang, Alexandra Gomez-Villa, Dipam Goswami, Xialei Liu, Joost van de Weijer, and Yonghong Tian. Continual learning for vlms: A survey and taxonomy beyond forgetting. *arXiv preprint arXiv:2508.04227*, 2025. 2

[25] Yuhang Liu, Pengxiang Li, Zishu Wei, Congkai Xie, Xueyu Hu, Xinchen Xu, Shengyu Zhang, Xiaotian Han, Hongxia Yang, and Fei Wu. Infiguiagent: A multimodal generalist

gui agent with native reasoning and reflection. *arXiv preprint arXiv:2501.04575*, 2025. 1, 2, 6

[26] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanjing Xiong, and Hongsheng Li. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning. *arXiv preprint arXiv:2503.21620*, 2025. 3

[27] Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*, 2025. 1, 2, 6

[28] Imad Eddine Marouf, Enzo Tartaglione, Stéphane Lathuiliere, and Joost van de Weijer. No images, no problem: Retaining knowledge in continual vqa with questions-only memory. *arXiv preprint arXiv:2502.04469*, 2025. 2

[29] Yuan Meng, Zhenshan Bing, Xiangtong Yao, Kejia Chen, Kai Huang, Yang Gao, Fuchun Sun, and Alois Knoll. Preserving and combining knowledge in robotic lifelong reinforcement learning. *Nature Machine Intelligence*, 7(2):256–269, 2025. 2

[30] Songqin Nong, Jingxuan Xu, Sheng Zhou, Jianfeng Chen, Xiaoxuan Tang, Tao Jiang, and Wenhao Xu. Craft-gui: Curriculum-reinforced agent for gui tasks. *arXiv preprint arXiv:2508.11360*, 2025. 3

[31] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025. 1, 2, 3, 6

[32] Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016. 2

[33] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025. 6

[34] Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, et al. Gui-g^2: Gaussian reward modeling for gui grounding. *arXiv preprint arXiv:2507.15846*, 2025. 3

[35] Mingyang Wang, Heike Adel, Lukas Lange, Jannik Strötgen, and Hinrich Schütze. Learn it or leave it: module composition and pruning for continual learning. *arXiv preprint arXiv:2406.18708*, 2024. 2

[36] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 1, 4

[37] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218*, 2024. 1, 2

[38] Donglai Xu, Hongzheng Yang, Yuzhi Zhao, Pingping Zhang, Jinpeng Chen, Wenao Ma, Zhijian Hou, Mengyang Wu, Xiaolei Li, Senkang Hu, et al. From exploration to exploitation: A two-stage entropy rlvr approach for noise-tolerant mllm training. *arXiv preprint arXiv:2511.07738*, 2025. 3

[39] Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. *arXiv preprint arXiv:1708.01547*, 2017. 2

[40] Yu-Chu Yu, Chi-Pin Huang, Jr-Jen Chen, Kai-Po Chang, Yung-Hsuan Lai, Fu-En Yang, and Yu-Chiang Frank Wang. Select and distill: Selective dual-teacher knowledge transfer for continual learning on vision-language models. In *European Conference on Computer Vision*, pages 219–236. Springer, 2024. 2

[41] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International conference on machine learning*, pages 3987–3995. PMLR, 2017. 2

[42] Xi Zhang, Feifei Zhang, and Changsheng Xu. Vqacl: A novel visual question answering continual learning setting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19102–19112, 2023. 2

[43] Yuze Zhao, Jintao Huang, Jinghan Hu, Xingjun Wang, Yunlin Mao, Daoze Zhang, Zeyinzi Jiang, Zhikai Wu, Baole Ai, Ang Wang, et al. Swift: a scalable lightweight infrastructure for fine-tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 29733–29735, 2025. 6

[44] Da-Wei Zhou, Yuanhan Zhang, Yan Wang, Jingyi Ning, Han-Jia Ye, De-Chuan Zhan, and Ziwei Liu. Learning without forgetting for vision-language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025. 2

[45] Yuqi Zhou, Sunhao Dai, Shuai Wang, Kaiwen Zhou, Qinglin Jia, and Jun Xu. Gui-g1: Understanding r1-zero-like training for visual grounding in gui agents. *arXiv preprint arXiv:2505.15810*, 2025. 3

[46] Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Hao Tian, Yuchen Duan, Weijie Su, Jie Shao, et al. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models. *arXiv preprint arXiv:2504.10479*, 2025. 1

[47] Zhen Zhu, Yiming Gong, and Derek Hoiem. Anytime continual learning for open vocabulary classification. In *European Conference on Computer Vision*, pages 269–285. Springer, 2024. 2